LLMs Unplugged

Understand AI by building it yourself

Speaker notes

Open with the promise: understand AI by building one yourself, by hand. No computers, no maths background needed.

Acknowledgement of Country

Speaker notes

Acknowledge the Traditional Custodians of the land you're meeting on, and pay respects to Elders past and present. Keep it brief and sincere; say it in your own words.

activity

everyone stand up

Speaker notes

Quick energiser. Read each line and let people sit; it speeds up, and by "the last 5 minutes" almost everyone's down. Lands the point that this stuff is already everywhere. ~2 min, don't linger.

sit down if you have

never used ChatGPT/Claude

sit down if you

haven’t used it in the last **month**

sit down if you

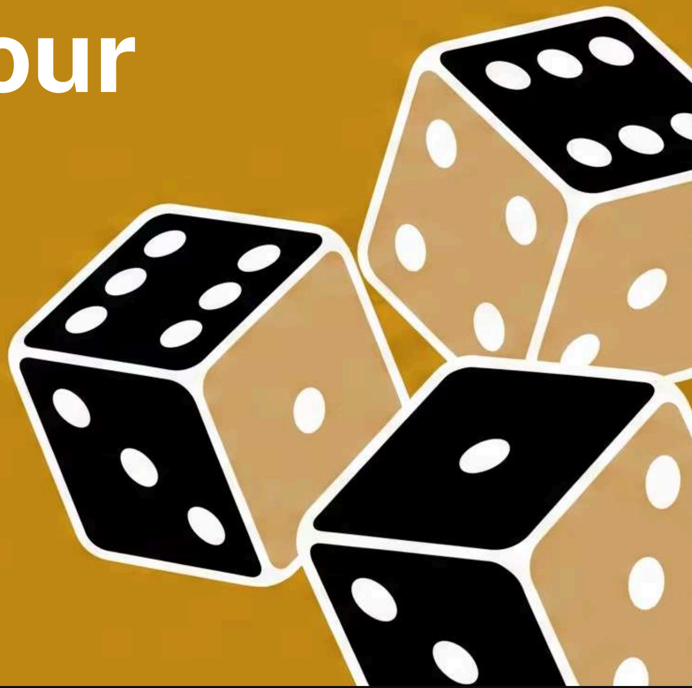
haven’t used it in the last **week**

sit down if you

haven’t used it in the last **day**

sit down if you

haven’t used it in the last **hour**



sit down if you

haven’t used it in the last **5 minutes**

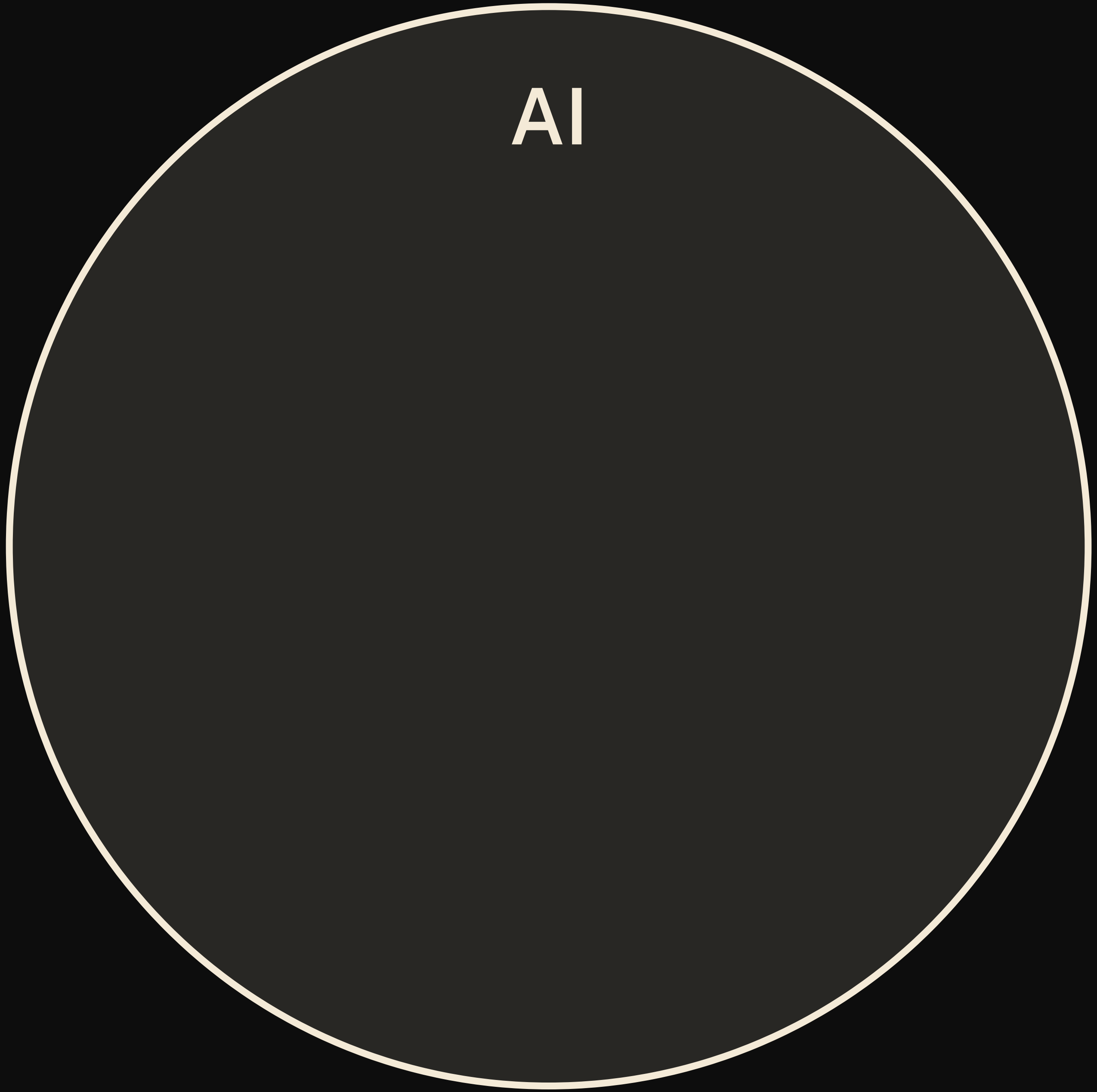
What is this about?

you'll **build** your own language model — from scratch — with just a kids book, pen & paper, and some dice rolling

you'll **learn** how language models work by spotting patterns in text to generate new text

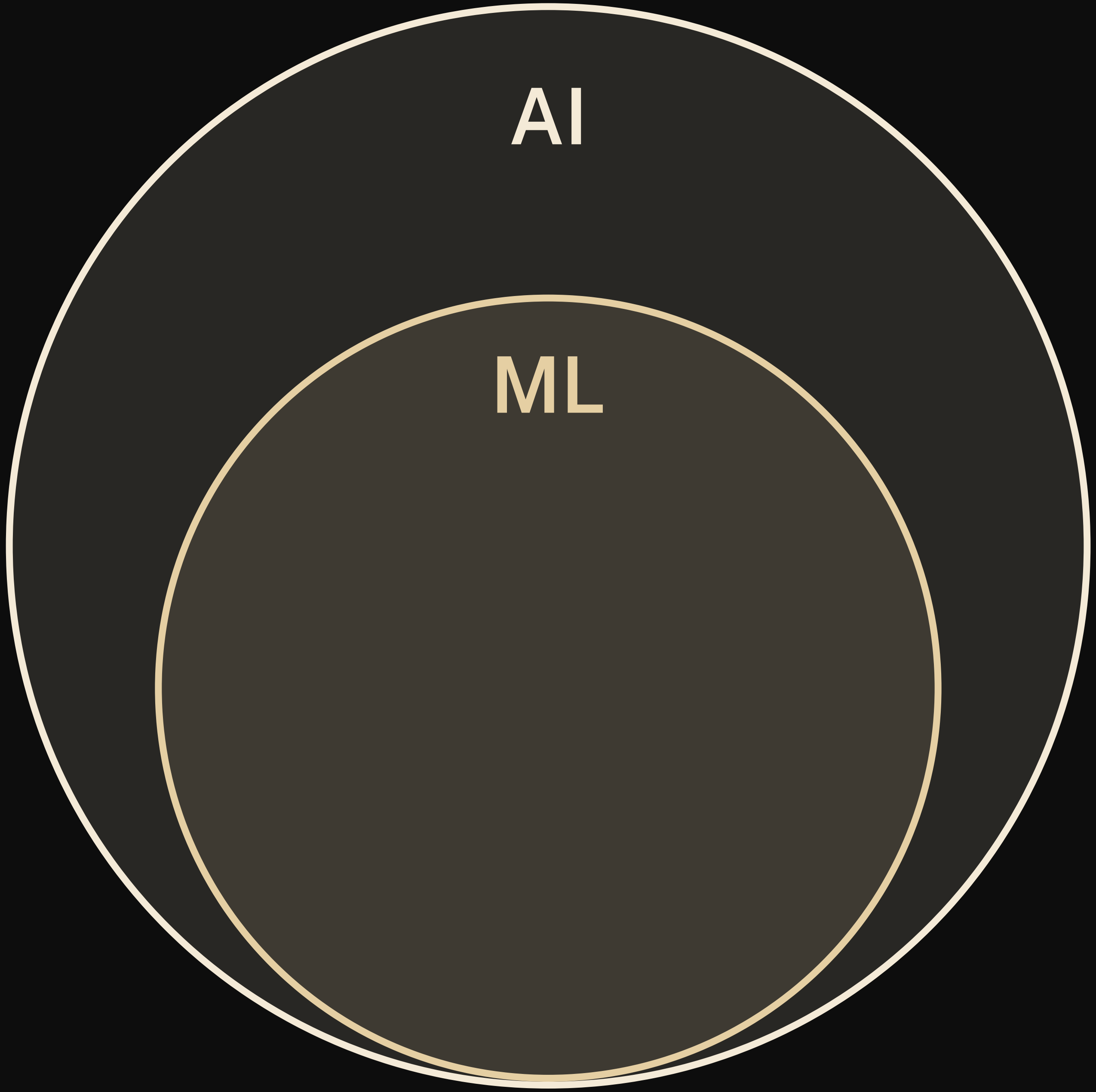
Speaker notes

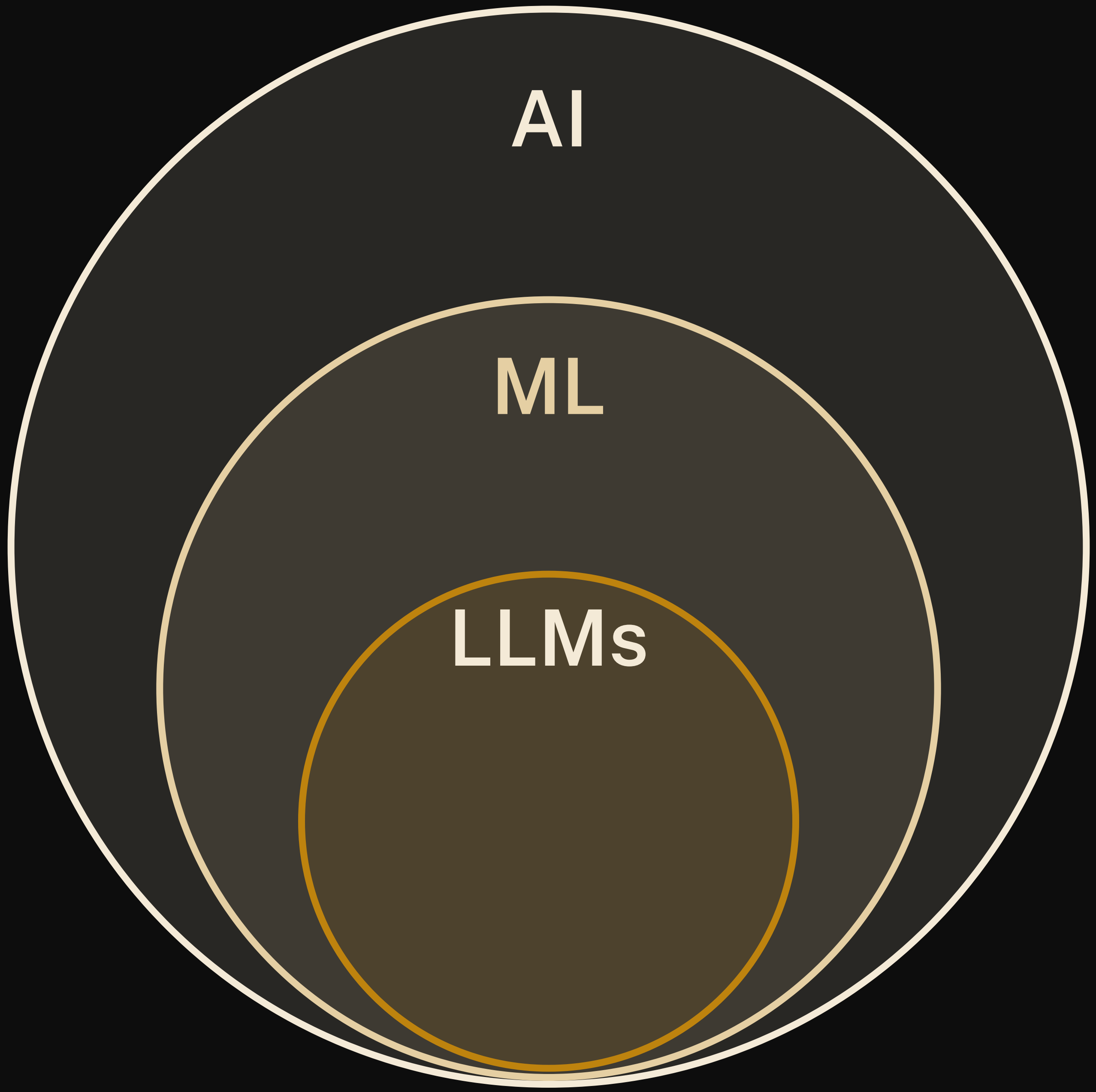
Frame the next stretch: build a model from a kids' book, pen, paper and dice, by spotting patterns. ~1 min, then move.

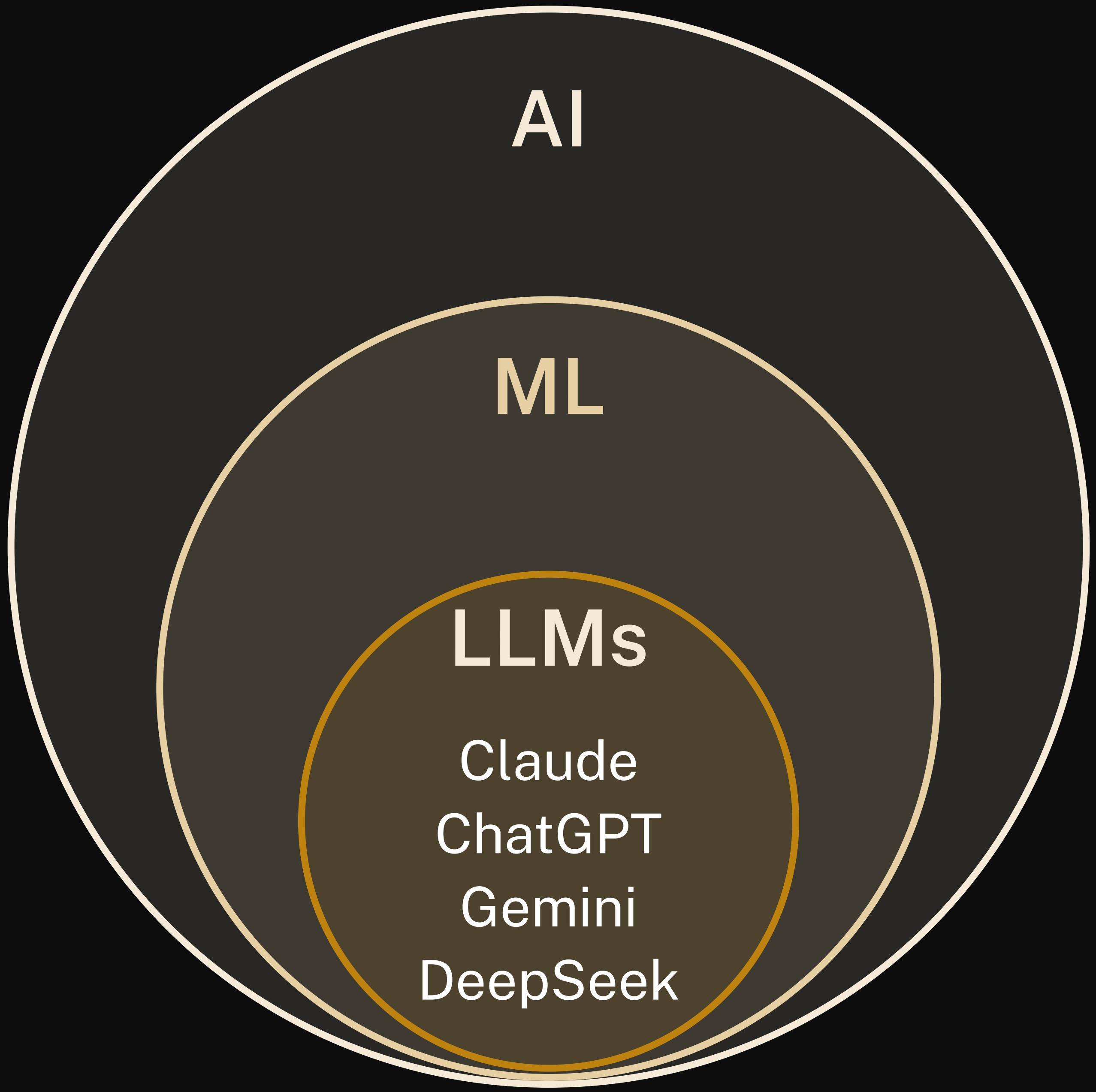

AI

Speaker notes

No text on this one --- talk over the build. AI: the umbrella term, any system doing things we'd call "smart" (chess engines, spam filters, route planners). Click --- ML: the subset that learns patterns from data instead of following hand-written rules; the model you'll build today lives here. Click --- LLMs: machine learning models trained on huge amounts of text to predict the next token --- exactly what you're about to do by hand, just scaled up. Click --- and the chatbots in the news (Claude, ChatGPT, Gemini, DeepSeek) are LLMs wrapped in an app. Same species as the thing you'll build with pen and dice.


AI
ML


AI
ML
LLMs

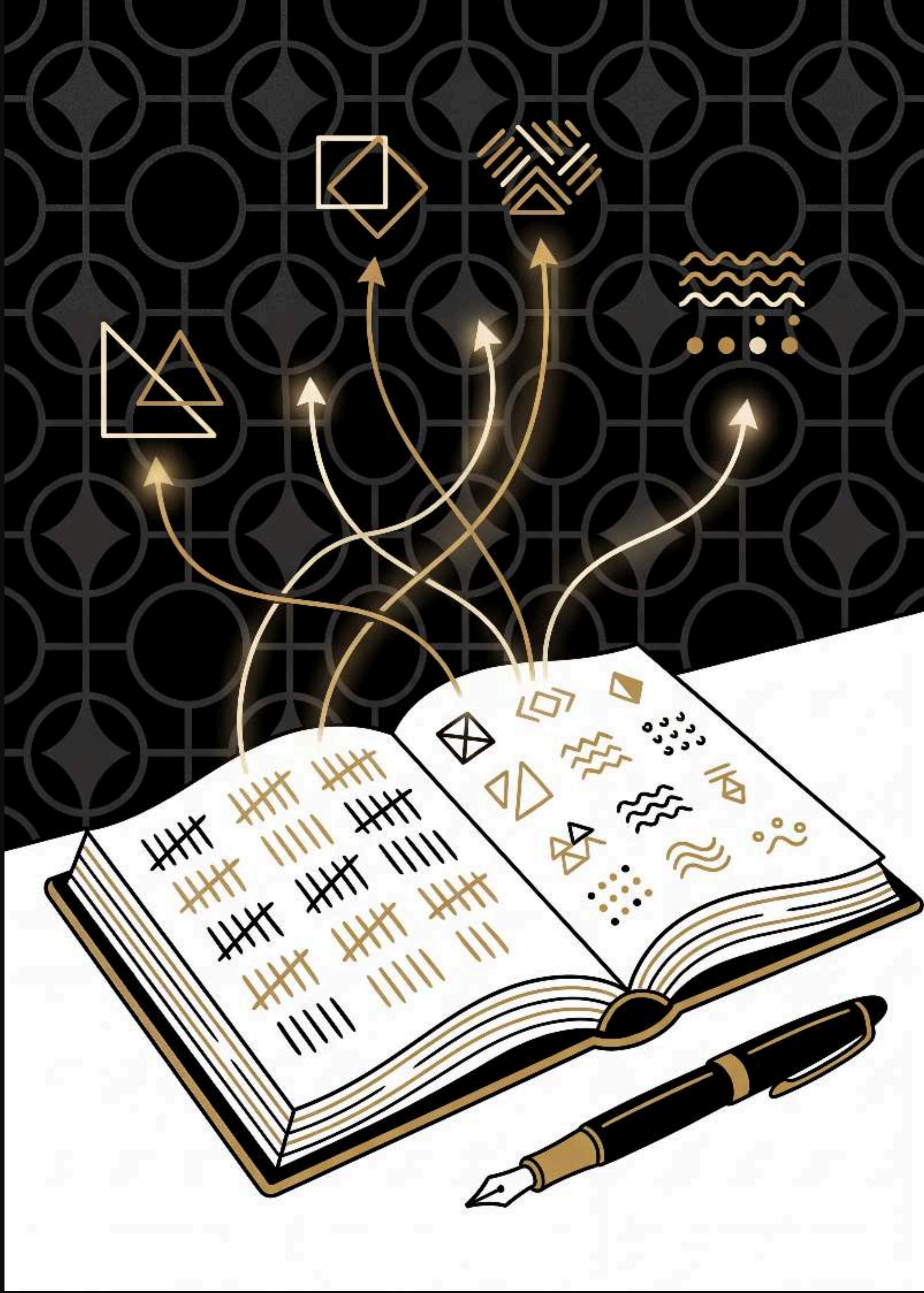

AI
ML
LLMs
Claude
ChatGPT
Gemini
DeepSeek

llmsunplugged.org

Training

The recipe

walk through your text and tally up which tokens follow which in a grid



Speaker notes

1. **tidy up** your text: lowercase everything, treat words and single punctuation marks (`. , ! ? ; :`) as separate tokens
2. **set up** your grid: first token goes in both the row & column headers
3. **fill in** the grid: for each consecutive pair of tokens, add a tally mark in that cell (add new rows/columns as needed)

Example

"Run, Spot, run. See Spot run."

after tidying up:

`run` `,` `spot` `,` `run` `.` `see` `spot` `run` `.`

The empty grid

run , spot , run . see spot run .

Training: run → ,

run , spot , run . see spot run .

	run	,			
run					
,					

Training: , → spot

run , spot , run . see spot run .

<table>
<tr><th></th><th>run</th><th>,</th><th>spot</th><th></th><th></th></tr>
<tr><td>run</td><td></td><td>|</td><td></td><td></td><td></td></tr>
<tr><td>,</td><td></td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>spot</td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td></tr>
</table>

Training: spot → ,

run , spot , run . see spot run .

	run	,	spot		
run		|			
,			|		
spot		|			

Speaker notes

, is already in our vocabulary — no new column needed!

Training: , → run

run , spot , run . see spot run .

	run	,	spot		
run		|			
,	|		|		
spot		|			

Training: run → .

run , spot , run . see spot run .

<table>
<tr><th></th><th>run</th><th>,</th><th>spot</th><th>.</th><th></th></tr>
<tr><td>run</td><td></td><td>|</td><td></td><td>|</td><td></td></tr>
<tr><td>,</td><td>|</td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>spot</td><td></td><td>|</td><td></td><td></td><td></td></tr>
<tr><td>.</td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td></tr>
</table>

Speaker notes

`.` is a new token — punctuation gets its own row and column too

Complete model

run , spot , run . see spot run .

<table>
<tr><th></th><th>run</th><th>,</th><th>spot</th><th>.</th><th>see</th></tr>
<tr><td>run</td><td></td><td>|</td><td></td><td>||</td><td></td></tr>
<tr><td>,</td><td>|</td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>spot</td><td>|</td><td>|</td><td></td><td></td><td></td></tr>
<tr><td>.</td><td></td><td></td><td></td><td></td><td>|</td></tr>
<tr><td>see</td><td></td><td></td><td>|</td><td></td><td></td></tr>
</table>

Speaker notes

with the rest of the text tallied the same way, the grid is complete. `run` → `.` came up twice, so its cell has two marks — those counts are what make some next-words more likely than others.

Training

10:00

start reset +1 −1

Speaker notes

Hand out grids; groups tally their own text. 10 min, circulate --- the usual snag is that a token only gets a new row and column the first time it appears.

The *language* of language models

- model
- token
- weights

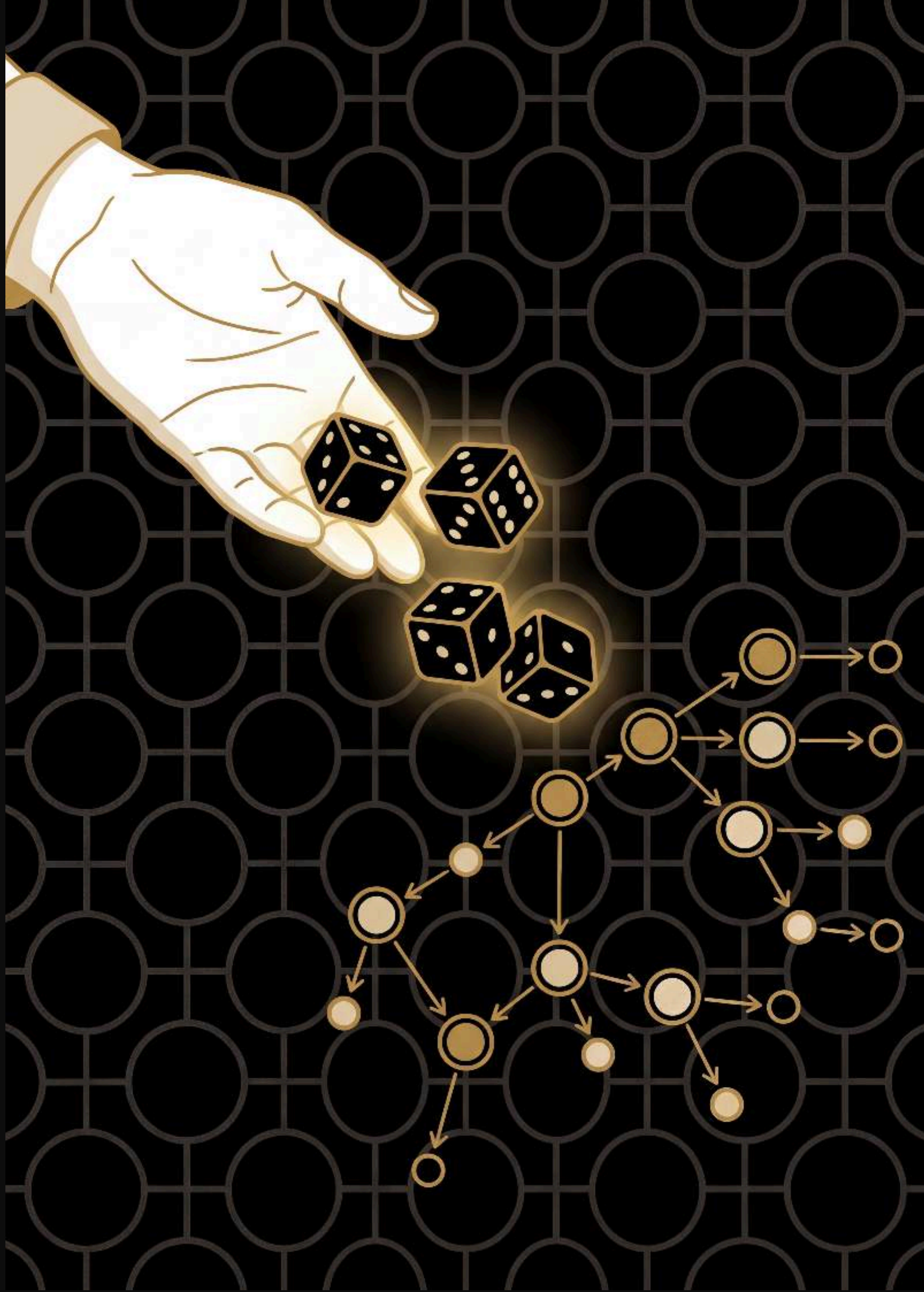
Speaker notes

Everyday sense first, then ours: a model is a fashion model or a model aeroplane --- here it's the thing that captures the patterns, your grid. A token is a bus token or a token gesture --- here it's one word or punctuation mark. Weights are what you lift at the gym, or what you weigh up before deciding --- here they're just the tally marks, the numbers that make some next words likelier than others. Point at a cell: when you hear a model has billions of weights, that's what's being counted.

Generation

The recipe

use your grid to generate new text, rolling dice to choose each next word



Speaker notes

1. **choose** a starting word from your grid (any word --- not just the first one)
2. **look** at that word's row to find all possible next words and their counts
3. **roll dice** weighted by the counts to pick the next word
4. **write down** the chosen word and make it your new starting word
5. **repeat** from step 2

Generation: start with see

see spot

one option — no roll needed

	run	,	spot	.	see
run		|		||	
,	|		|		
spot	|	|			
.					|
see			|		

Speaker notes

any word will do as a seed---we're starting from `see`, which is the last row, to show you don't have to begin at the top. Only one option here, so no roll.

Generation: from spot

see spot

2 options — roll the die!

<table>
<tr><th></th><th>run</th><th>,</th><th>spot</th><th>.</th><th>see</th></tr>
<tr><td>run</td><td></td><td>|</td><td></td><td>||</td><td></td></tr>
<tr><td>,</td><td>|</td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>spot</td><td>|</td><td>|</td><td></td><td></td><td></td></tr>
<tr><td>.</td><td></td><td></td><td></td><td></td><td>|</td></tr>
<tr><td>see</td><td></td><td></td><td>|</td><td></td><td></td></tr>
</table>

Speaker notes

two options in this row (`run` and `,`) with equal tallies --- highlight both, then roll

How the die chooses: `spot`

`spot` → ? roll a d10

0	1	2	3	4	5	6	7	8	9

`run`
1 tally

`,`
1 tally

equal tallies → equal chances

Speaker notes

equal tallies, so each option gets an equal share of the ten d10 faces --- `run` gets 0-4 and `,` gets 5-9, reading the row's cells left to right. Pause here before rolling.

How the die chooses: `spot`

`spot` → ? roll a d10

0	1	2	3	4	5	6	7	8	9

`run`
1 tally

`,`
1 tally

equal tallies → equal chances

rolled 7 → `,`

Speaker notes

now roll --- a 7 lands in the `,` band

Generation: spot → ,

see spot ,

rolled 7 → ,

	run	,	spot	.	see
run		|		||	
,	|		|		
spot	|	|			
.					|
see			|		

Speaker notes

the 7 landed in the `,` band, so `,` is our next word

Generation: from ,

see spot ,

2 options — roll the die!

<table>
<tr><th></th><th>run</th><th>,</th><th>spot</th><th>.</th><th>see</th></tr>
<tr><td>run</td><td></td><td>|</td><td></td><td>||</td><td></td></tr>
<tr><td>,</td><td>|</td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>spot</td><td>|</td><td>|</td><td></td><td></td><td></td></tr>
<tr><td>.</td><td></td><td></td><td></td><td></td><td>|</td></tr>
<tr><td>see</td><td></td><td></td><td>|</td><td></td><td></td></tr>
</table>

Speaker notes

another two-option row (`run` and `spot`), still equal --- both highlighted, then roll

Generation: , → run

see spot , run

rolled **2** → run

	run	,	spot	.	see
run		$\vert$		$\Vert$	
,	$\vert$		$\vert$		
spot	$\vert$	$\vert$			
.					$\vert$
see			$\vert$		

Speaker notes

rolled a 2, which lands in the `run` band

Generation: from run

see spot , run

2 options — roll the die!

	run	,	spot	.	see
run		|		||	
,	|		|		
spot	|	|			
.					|
see			|		

Speaker notes

two options again --- but `.` has two tallies to `,`'s one, so this isn't a 50/50 roll

How the die chooses: `run`

`run` → ? roll a d10

0	1	2	3	4	5	6	7	8	9

`,` 1 tally

`.` 2 tallies

more tallies → more faces → more likely

Speaker notes

`.` has twice the tallies, so it gets more of the faces (3-9) than `,` (0-2). With a ten-sided die that's the closest split rather than exactly 2:1---the point is more tallies → more faces → more likely. Pause here before rolling.

How the die chooses: `run`

`run` → ? roll a d10

0	1	2	**3**	4	5	6	7	8	9

`,` 1 tally

`.` 2 tallies

more tallies → more faces → more likely

rolled **3** → `.`

Speaker notes

now roll --- a 3 sits in the `.` band

Generation: run → .

see spot , run .

rolled 3 → .

<table>
<tr><th></th><th>run</th><th>,</th><th>spot</th><th>.</th><th>see</th></tr>
<tr><td>run</td><td></td><td>|</td><td></td><td>||</td><td></td></tr>
<tr><td>,</td><td>|</td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>spot</td><td>|</td><td>|</td><td></td><td></td><td></td></tr>
<tr><td>.</td><td></td><td></td><td></td><td></td><td>|</td></tr>
<tr><td>see</td><td></td><td></td><td>|</td><td></td><td></td></tr>
</table>

Speaker notes

the 3 sits in the `.` band, so we pick `.`

Generation: from .

see spot , run . see

one option — no roll needed

	run	,	spot	.	see
run		|		||	
,	|		|		
spot	|	|			
.					|
see			|		

Speaker notes

only one option (`see`) --- no roll. Note we keep rolling straight past the full stop; the model has no idea a sentence just ended.

Generation: back to see

see spot , run . see

one option — no roll needed

<table>
<tr><th></th><th>run</th><th>,</th><th>spot</th><th>.</th><th>see</th></tr>
<tr><td>run</td><td></td><td>|</td><td></td><td>||</td><td></td></tr>
<tr><td>,</td><td>|</td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>spot</td><td>|</td><td>|</td><td></td><td></td><td></td></tr>
<tr><td>.</td><td></td><td></td><td></td><td></td><td>|</td></tr>
<tr><td>see</td><td></td><td></td><td>|</td><td></td><td></td></tr>
</table>

Speaker notes

and so on --- we've looped back to where we started. Keep going for as long as you like; you decide when to stop.

Generated text

“see spot, run. see”

a new sentence — not in the training data!

10:00

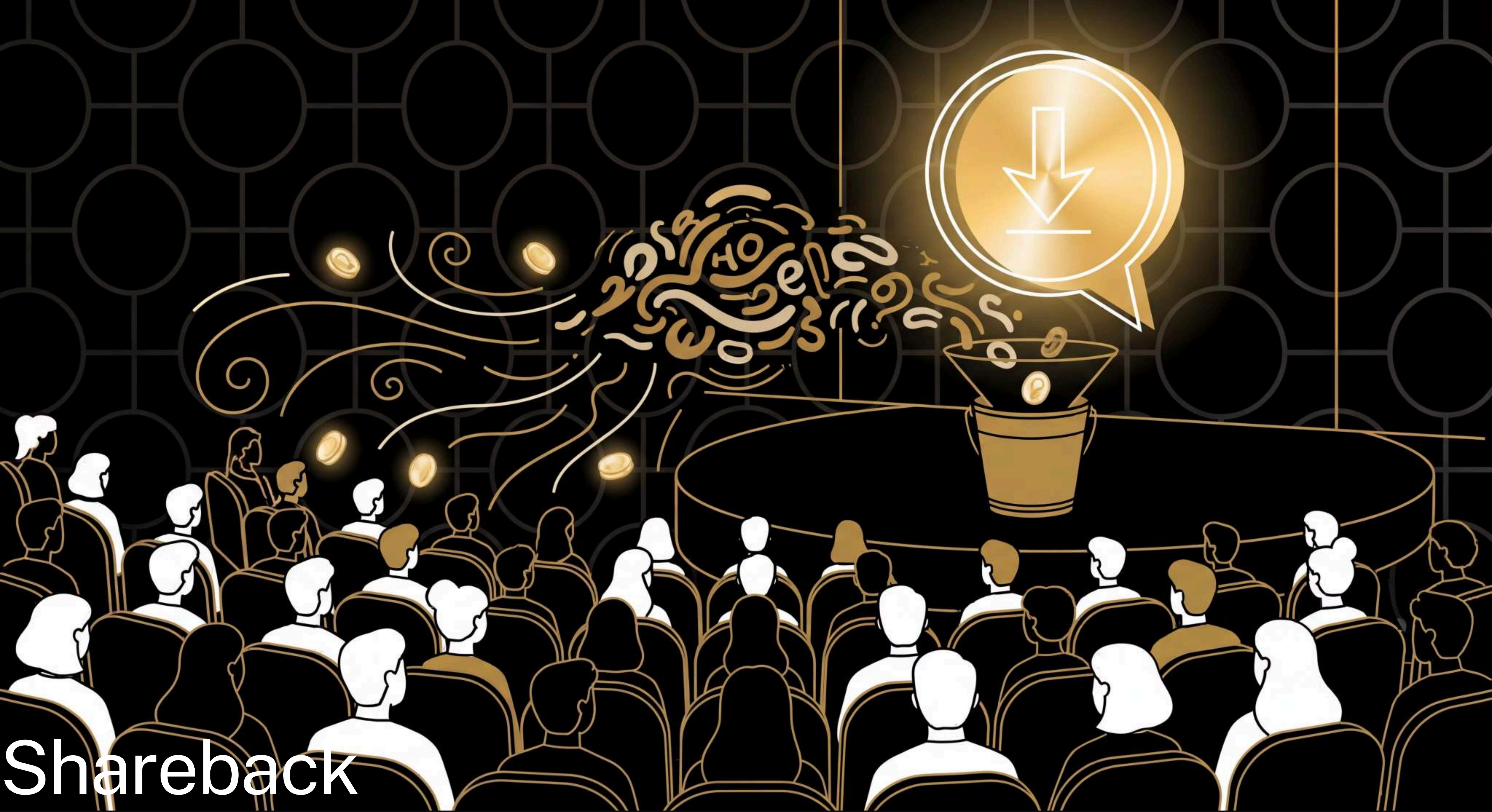
Generation

start reset +1 −1

Speaker notes

Groups generate from their grid, rolling dice on each choice. 10 min, circulate. Where a row has only one option, no roll needed.

Shareback



Speaker notes

Take a couple of groups' generated sentences; celebrate the weird ones. ~5 min.

The *language* of language models

- prompt
- completion
- inference
- hallucination

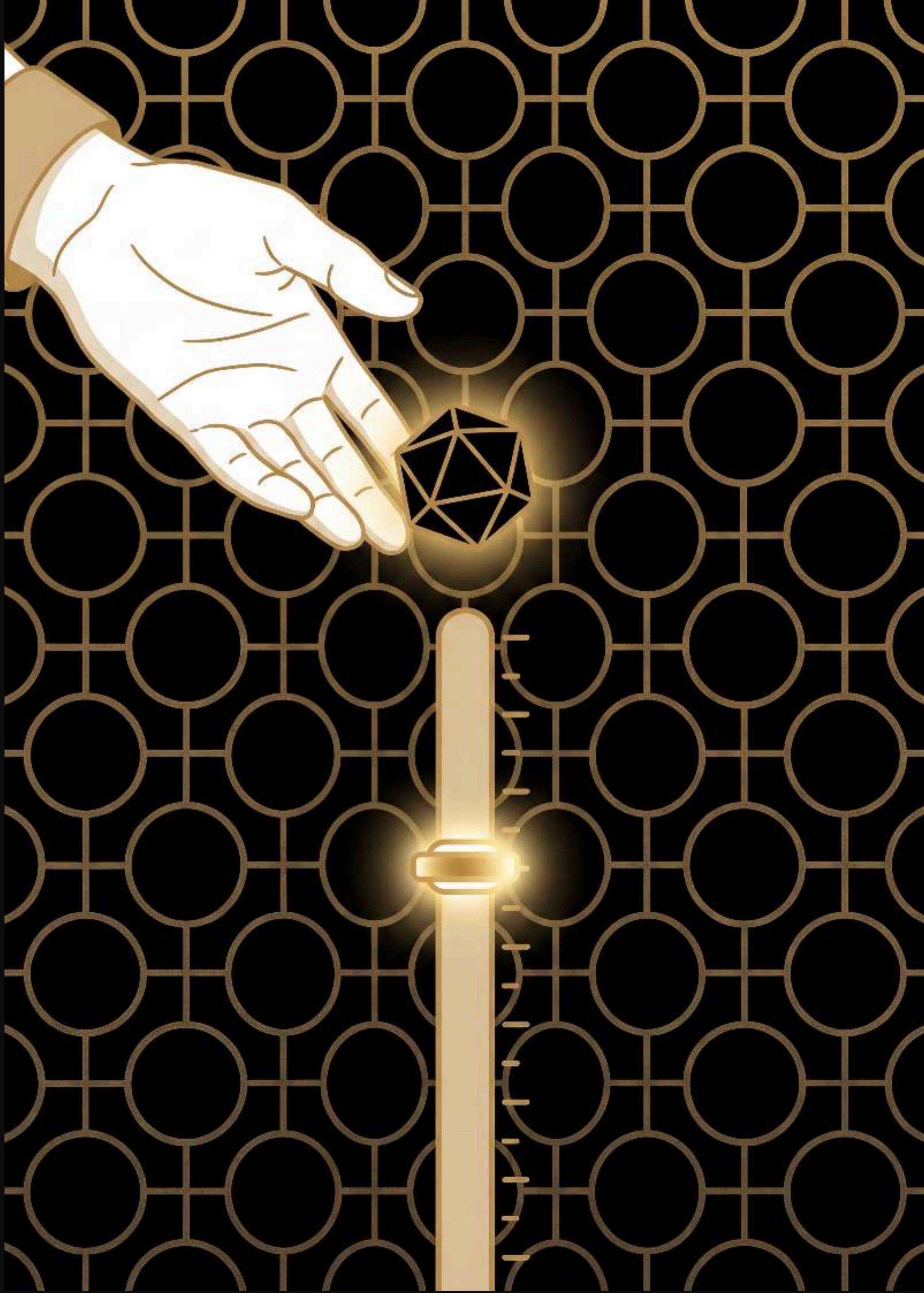
Speaker notes

Everyday sense first, then ours: to prompt is to nudge an actor who's forgotten a line --- here the prompt is the starting text you give the model. A completion is finishing something off --- here it's the text the model generates back, and it doesn't know when to stop. An inference is a conclusion you work out from evidence, what a detective does --- here it's just the name for running the model forward, the rolling and writing you've been doing for the last ten minutes: training builds the grid, inference uses it. A hallucination is seeing something that isn't there, a malfunction --- but the sentence you just generated was never in the text either, and it came out of the same tallies and the same dice as everything else. There's no separate mode the model slips into; we only call it hallucination when we didn't want the answer.

Pre-trained generation

The recipe

use the booklet to generate new text, rolling one or more d10s to choose each next word



Speaker notes

1. **choose** a starting word --- any bold word in the booklet
2. **look up** that word's entry to see possible next words and their thresholds
3. **roll** your d10(s) and scan the thresholds to find the next word
4. **write down** the chosen word and make it your new starting word
5. **repeat** from step 2

Pre-trained: start with the

the cat

the ♦♦ 49|cat 79|dog 99|hat **rolled 27**

cat ♦ 5|sat 9|ran

sat .

. the

ran .

dog ♦ 6|sat 9|ran

hat ♦ 4|sat 9|ran

Speaker notes

a bigger model — `the` is so common it needs **two** dice (roll two d10s for 00 to 99)

Pre-trained: from `cat`

`the` `cat` `sat`

the ♦♦ 49|cat 79|dog 99|hat

cat ♦ 5|sat 9|ran **rolled 4**

sat .

. the

ran .

dog ♦ 6|sat 9|ran

hat ♦ 4|sat 9|ran

Speaker notes

`cat` is rarer than `the` — just **one** die, roll a d10

Pre-trained: from `sat`

`the` `cat` `sat` `.`

the ♦♦ 49|cat 79|dog 99|hat

cat ♦ 5|sat 9|ran

sat .

. the

ran .

dog ♦ 6|sat 9|ran

hat ♦ 4|sat 9|ran

Speaker notes

only one option — no dice roll needed

Pre-trained: from .

the cat sat . the

the ♦♦ 49|cat 79|dog 99|hat

cat ♦ 5|sat 9|ran

sat .

. the

ran .

dog ♦ 6|sat 9|ran

hat ♦ 4|sat 9|ran

Speaker notes

only one option — no dice roll needed

Pre-trained: from the again

the cat sat . the dog

the ♦♦ 49|cat 79|dog 99|hat **rolled 63**

cat ♦ 5|sat 9|ran

sat .

. the

ran .

dog ♦ 6|sat 9|ran

hat ♦ 4|sat 9|ran

Speaker notes

two dice again — 63 is past `cat`'s 49, so we land on `dog`

Pre-trained: from dog

the cat sat . the dog

the ♦♦ 49|cat 79|dog 99|hat

cat ♦ 5|sat 9|ran

sat .

. the

ran .

dog ♦ 6|sat 9|ran

hat ♦ 4|sat 9|ran

Speaker notes

and so on...

Pre-trained generation

8:00

start reset +1 −1

Speaker notes

Same as before but from the booklet --- a bigger model, one or more d10s. 8 min, circulate.

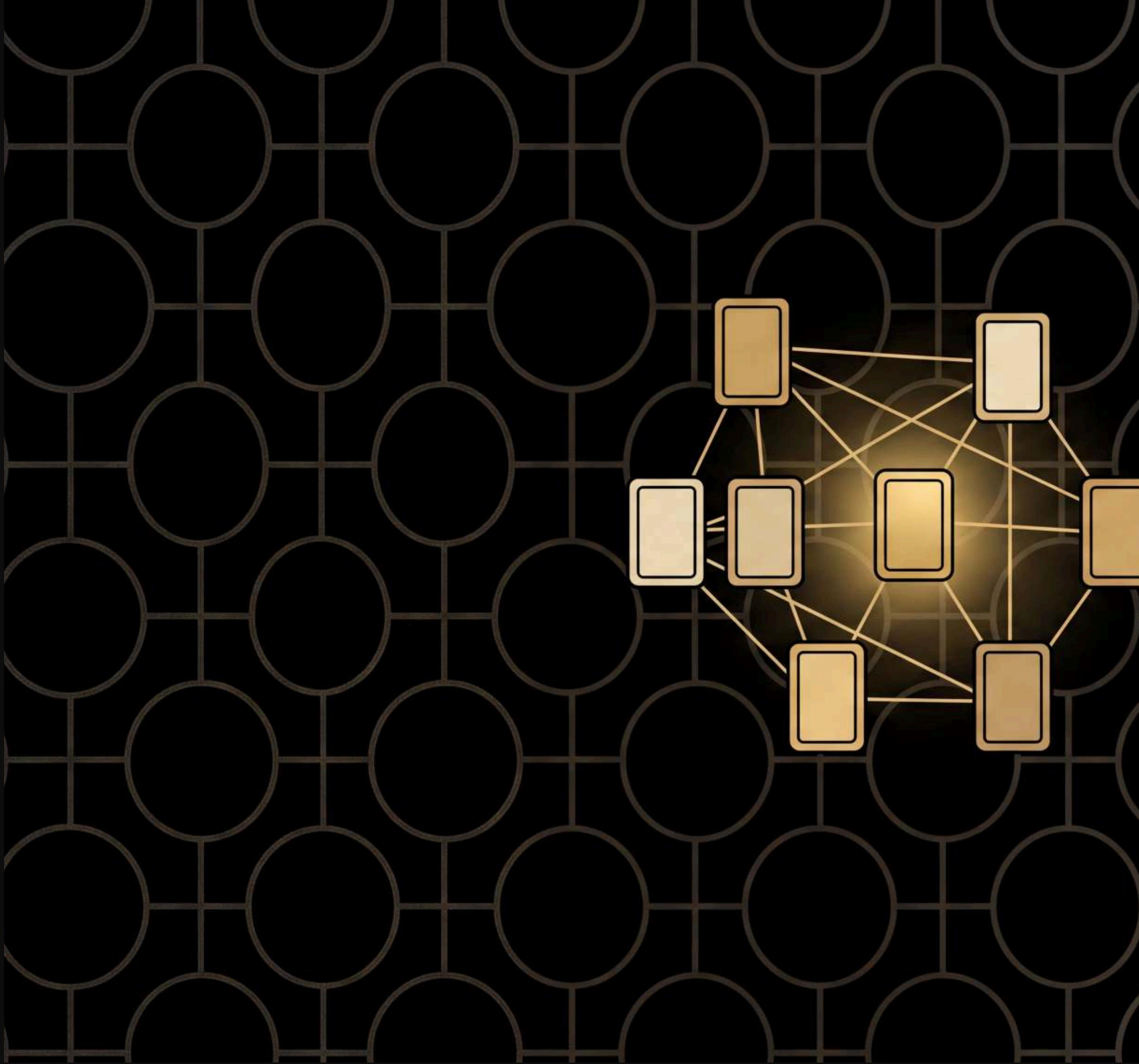
Shareback

Speaker notes

Compare with the hand-built model: a bigger training text gives more varied, more natural output. Invite ~5 groups to share back rather than the whole room. ~5 min.

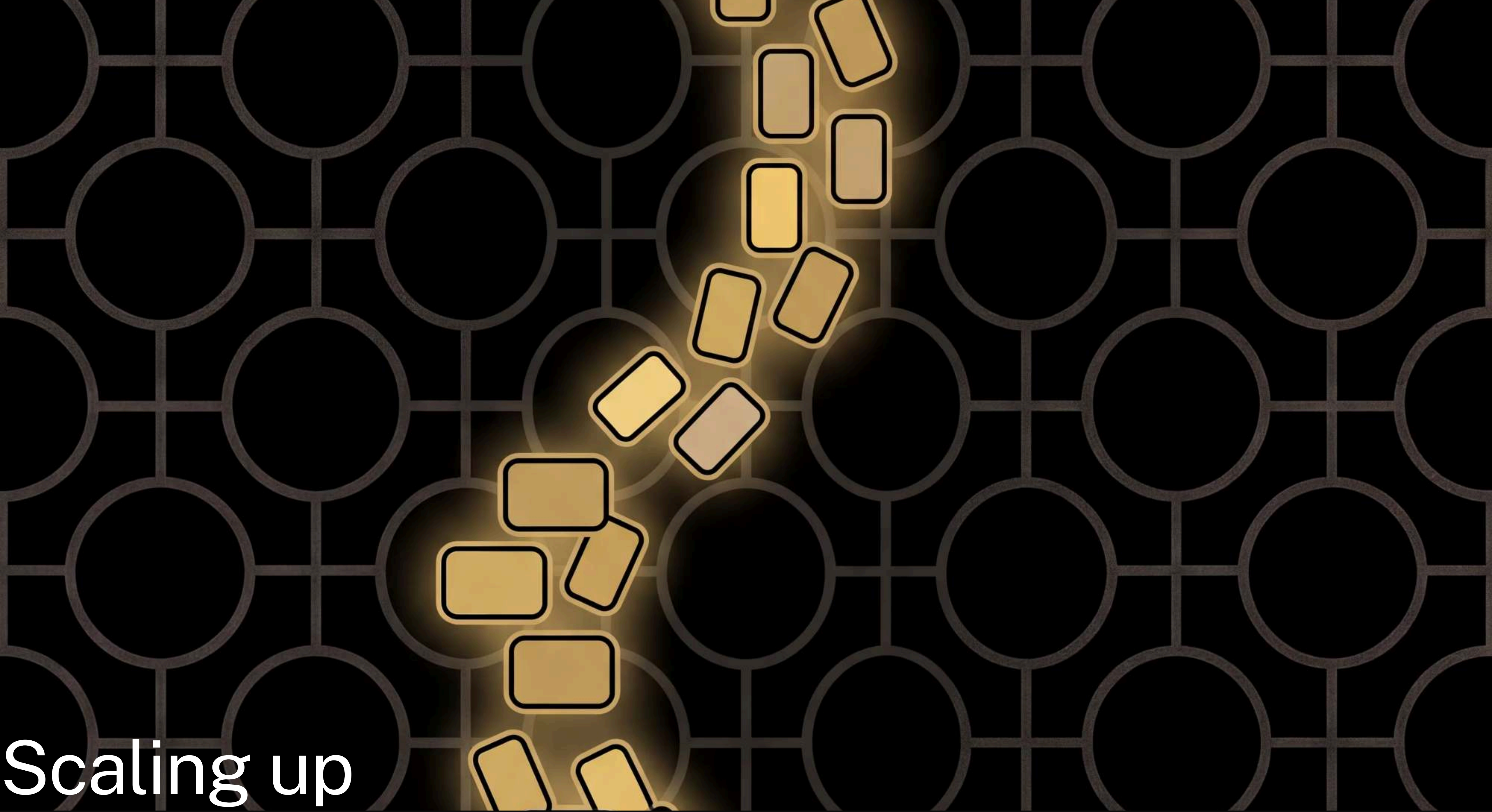
The *language* of language models

- foundation model



Speaker notes

Everyday sense first, then ours: a foundation is what a house sits on, or a charity --- here a foundation model is the general-purpose thing you get from one enormous training run, which everything else is built on. Worth saying that "pre-training" sounds like a warm-up but is the main event; the booklet they've just used is the output of one.



Scaling up

Speaker notes

The reassurance beat: what they built isn't a cartoon of an LLM, it's the real mechanism at human scale---look at the context, weigh every possible next word, roll the dice, write it down, repeat. So what's actually different? Three differences, and then scale. Each one is run against their own grid rather than as a separate diagram: make the grid fail at something on stage, then say what a real model does instead. Resist the urge to explain the machinery---this section is the conceptual shape, and every one of these has a lesson on the website that does it properly. ~7 min.

One word of context

see spot ,

2 options — roll the die!

<table>
<tr><th></th><th>run</th><th>,</th><th>spot</th><th>.</th><th>see</th></tr>
<tr><td>run</td><td></td><td>|</td><td></td><td>||</td><td></td></tr>
<tr><td>,</td><td>|</td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>spot</td><td>|</td><td>|</td><td></td><td></td><td></td></tr>
<tr><td>.</td><td></td><td></td><td></td><td></td><td>|</td></tr>
<tr><td>see</td><td></td><td></td><td>|</td><td></td><td></td></tr>
</table>

your grid sees **one word**; Claude sees **a million tokens** — the whole conversation, every document in it

Speaker notes

First difference, and it's sitting right there in the worked example. We're on a comma with two options and no way to choose between them, so we roll. Look at why we're stuck: the model is looking at a comma, and every comma in the text is identical to it. It cannot tell whether it just wrote `run` or `spot`. That's not bad luck, it's the design --- one word of context is all it gets. A frontier model looks back over everything it can see, and that's the whole conversation plus whatever you pasted into it. If someone asks the obvious "why not just look at two words?" --- you can, that's a trigram, and there's a lesson on the website that runs one with pen and paper. But the grid needs a row per context, so each extra word multiplies it by the size of the vocabulary: 5, 25, 125, and at a real vocabulary of ~100,000 you run out of atoms in the universe before you run out of sentence. Real models don't store a row per context, they work out what to pay attention to on the fly. Don't go further than that here.

cat and dog are strangers

the cat sat . the cat ran . the dog sat .

<table>
<tr><th></th><th>the</th><th>cat</th><th>sat</th><th>.</th><th>ran</th><th>dog</th></tr>
<tr><td>the</td><td></td><td>||</td><td></td><td></td><td></td><td>|</td></tr>
<tr><td>cat</td><td></td><td></td><td>|</td><td></td><td>|</td><td></td></tr>
<tr><td>sat</td><td></td><td></td><td></td><td>||</td><td></td><td></td></tr>
<tr><td>.</td><td>||</td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>ran</td><td></td><td></td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>dog</td><td></td><td></td><td>|</td><td></td><td></td><td></td></tr>
</table>

Speaker notes

Second difference, and it starts with something the grid is good at: everything it knows is somewhere you can put your finger on. Hold that thought while you read this text. `cat` and `dog` do exactly the same job here --- both come after `the`, both get followed by verbs. But look at the rows: `cat` has seen `sat` and `ran`; `dog` has only seen `sat`. Ask this grid to continue "the dog" and it will never, ever say `ran`, because that cell is empty and an empty cell means impossible. You know "the dog ran" is fine. The grid has no way to know, because nothing it learned about `cat` is connected to `dog` in any way. Every cell is its own little island.

Tallies become knobs

	the	cat	sat	.	ran	dog
the	0.03	0.46	0.04	0.02	0.04	0.41
cat	0.04	0.01	0.48	0.03	0.42	0.02
sat	0.12	0.02	0.01	0.78	0.04	0.03
.	0.82	0.05	0.03	0.02	0.04	0.04
ran	0.11	0.03	0.03	0.77	0.02	0.04
dog	0.05	0.02	0.46	0.03	0.40	0.04

a number in every cell — even the one your grid left empty — because it is **billions of knobs** doing the remembering, not cells

Speaker notes

The fix, on the same grid---same text, same rows, the tally marks ghosted in place with a number under each. Two things to point at. First, the ringed cell: `dog` followed by `ran`, empty in the tallies, 0.40 here. Second, the `cat` and `dog` rows, which have come out nearly identical, because in this text the two words do the same job. A transformer doesn't give each pattern its own cell; it spreads every pattern across billions of numbers, each nudged a tiny amount after every wrong guess in training. Because `cat` and `dog` turn up in the same kinds of places, training pushes them onto overlapping numbers---so what the model learns from `cat` is already, for free, something it knows about `dog`. That's generalisation, and it's the thing the grid structurally cannot do. The cost is the flip side of the slide before: there's no longer a cell to point at. Ask where a transformer stores "the dog ran" and the honest answer is "smeared across a few billion numbers, and we're still working on reading it back". That's most of what interpretability research is.

prompt: “what is the capital of France?”

your grid `“see spot, run. see”`

it isn’t refusing to answer — **answering isn’t a thing it does**

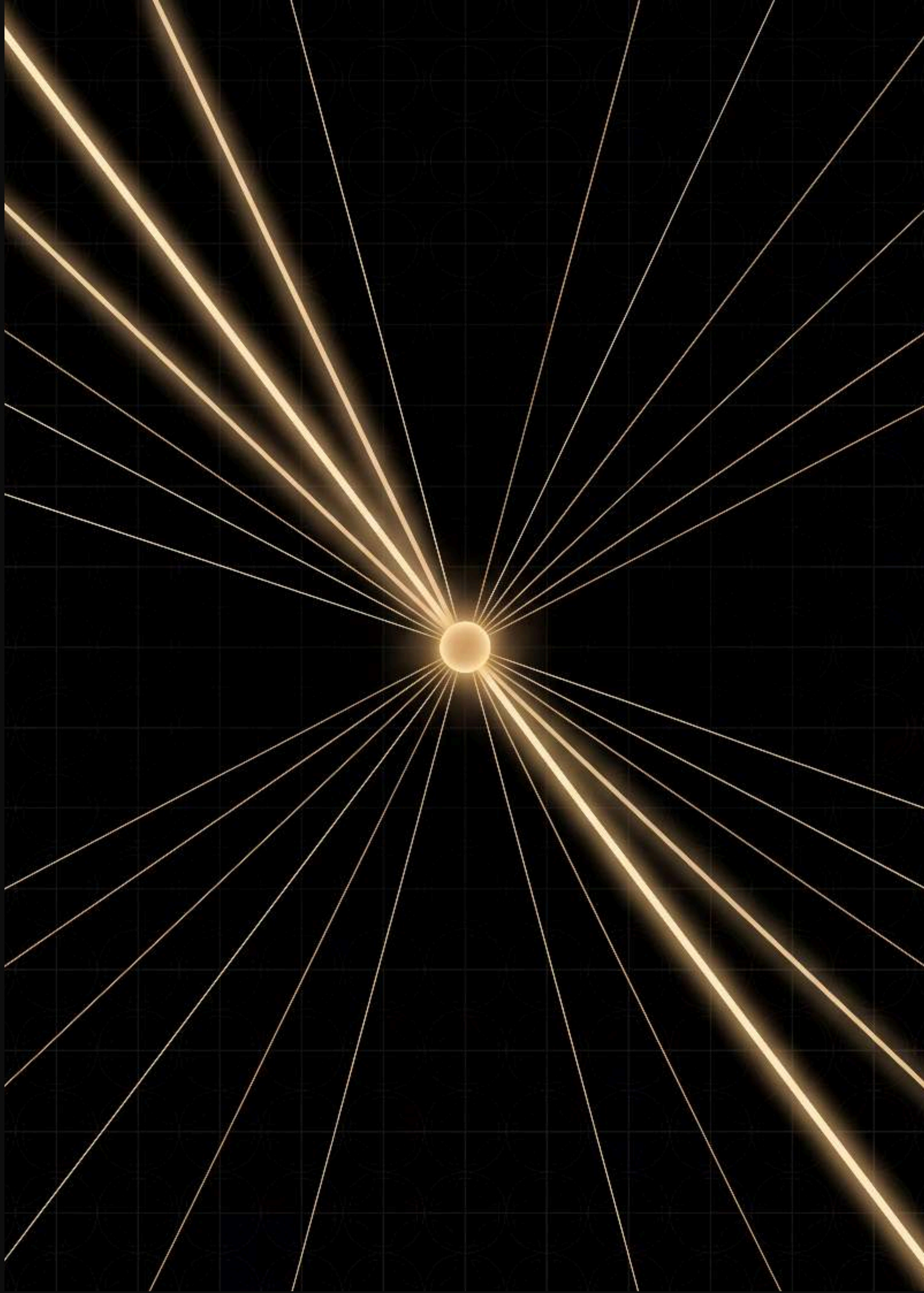
the fix is more training, on text made of *conversations*

Speaker notes

No heading --- the line is "let's ask your grid a question", and the slide just shows what comes back. Third difference, and the funniest one. Feed their model a question and it does what it always does: continues. It isn't being unhelpful and it isn't broken; nothing in "tally which word follows which" has any notion of a question and an answer. This is the honest state of a model that has only ever been trained to continue text --- and it's also, near enough, what a frontier model looks like straight out of pre-training: their problem, with better grammar. Then the second line. What turns it into something that answers you is post-training: a second, much smaller round of the same training on transcripts of conversations, plus feedback on which answers people preferred. Tally text off the web and what follows a question mark is usually another question; tally conversations and what follows it is the start of an answer. The mechanism hasn't changed --- same predict-the-next-word loop, same dice --- just different text to learn from, so different habits on top. Don't labour it; the joke has already made the point.

Three things are different

- it sees **more than one word**
- it learns **patterns**, not tallies
- it's been taught to **answer**, not just continue



Speaker notes

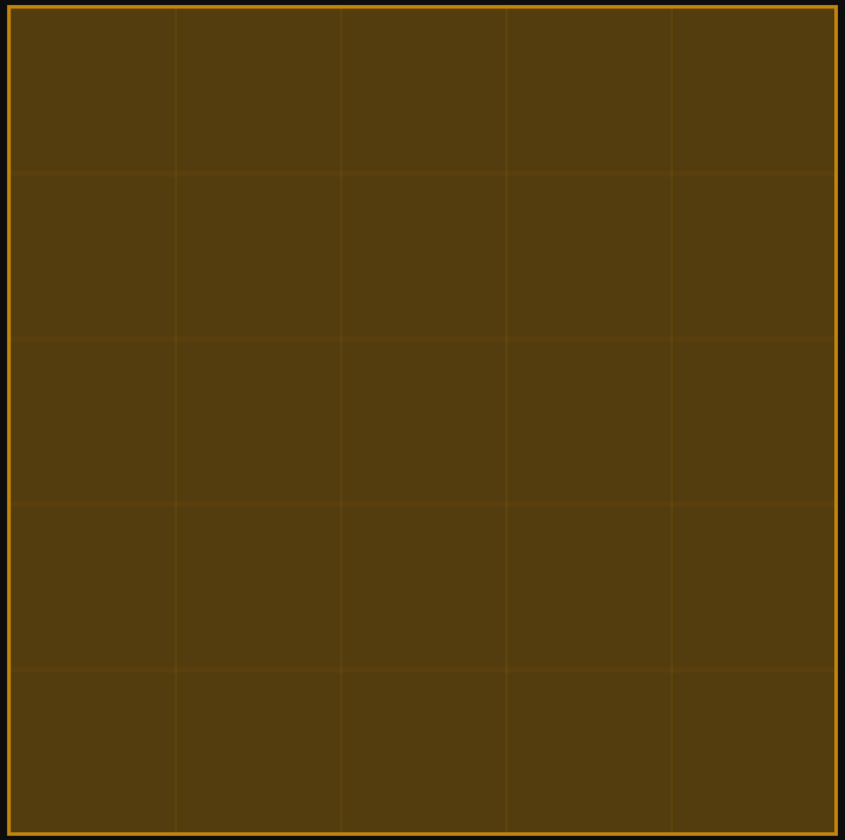
Land the three before the last one, because the last one isn't a difference in kind at all. None of these changed the loop: look at the context, weigh the options, roll, write it down, repeat. What's left is dosage --- and that's the one people's intuition has no grip on.

Scale is a hell of a drug

Speaker notes

Keep your eye on the gold square over the next two slides: that's their grid, drawn at true scale, on both of them.

your grid

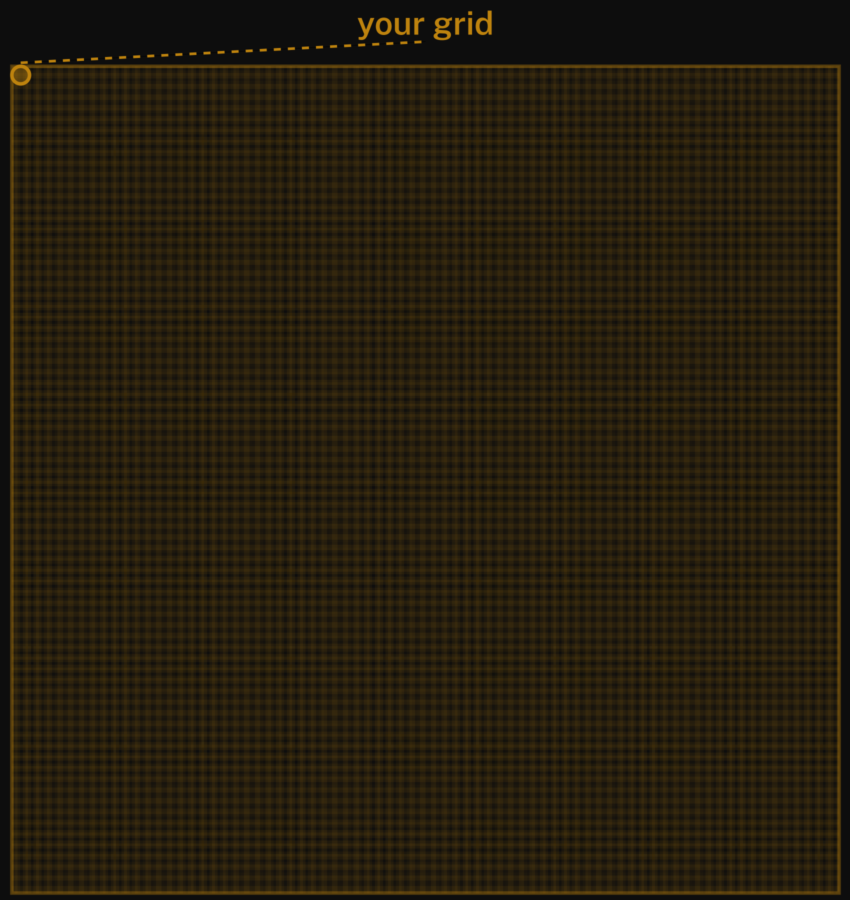


“Run, Spot, run. See Spot run.”

5 tokens · 25 cells

Speaker notes

Figure-only, so you carry it: this is "your grid". Five tokens, twenty-five cells. At the booklet's 1cm per cell that's a five-centimetre square --- smaller than a beer coaster.



Frankenstein

7,441 tokens · 55 million cells

the cells are now too small to draw

Speaker notes

Same again, no heading --- read the caption out. Frankenstein, the novel in the booklet: 7,441 distinct tokens, 55 million cells. The grid lines have stopped being drawable, so that grey *is* the grid. On paper it's about 5,500 square metres, most of a football field, and their model is the gold speck. This is also why the booklet doesn't print a grid: at this size you list only the cells that aren't empty. Note it's the vocabulary that matters here, not the length --- a novel reuses its words relentlessly.

a real vocabulary: **100,000** tokens, **10 billion** cells, **1 km^2** of paper

Speaker notes

Stop drawing and start measuring, because the picture gives out here. A commercial model's vocabulary is around 100,000, so its bigram grid alone is 10 billion cells---a square kilometre of paper, near enough the whole Acton campus. And that is still the toy version: one word of context, tally marks, none of the three fixes we just went through. Two more ballparks if you want them. The real thing: a frontier model's ~1.5 trillion parameters at 1cm each is about 150 km^2, a square 12km on a side, a hundred Acton campuses, twenty-odd Lake Burley Griffins. And time: if hand-training 100 tokens took you 10 minutes, the tens of trillions of tokens these are trained on would take about 5.7 million years at your rate---you'd have had to start around when our ancestors split from the chimpanzees to be finishing now. The punchline isn't any one figure. It's that the mechanism didn't change, only the dosage.

still just **tokens in → tokens out**

Speaker notes

Every reply from Claude or ChatGPT is sampled one word at a time, exactly like their dice rolls --- that's why "regenerate" gives a different answer. More context, patterns instead of tallies, a round of manners --- but the loop is the one they ran by hand.

Questions

what new questions do you have about large language models?

Speaker notes

Open the floor on the first question and let it run. Bring up the second only once the questions thin out --- it turns the session outward, from what the grid does to what they'll do differently on Monday.

Questions

what new questions do you have about large language models?

how will this change the way you think about and use LLMs in the future?

Australian National University

School of Cybernetics

www.llmsunplugged.org